

Design Note: Offline Satellite Tile Classification Service

Overview: A single offline service that takes a satellite tile, classifies it into one of 7 land-use classes (Forest, River, Residential, Industrial, AnnualCrop, SeaLake, Highway) on CPU, and stores the result with enough context to debug and re-evaluate it. Every prediction is stored, but low-confidence ones are flagged for review instead of being silently trusted.

Architecture and tile flow:

Four components, one process: a FastAPI app (ingest + query), a classifier module loaded once at startup, a SQLite database, and a local folder for raw tiles. No network calls at runtime.

Flow: tile uploaded -> validate -> Hash and dedupe -> preprocess -> classify -> confidence policy -> SQLite (storage) -> query API

1. Validate: reject empty files, files over 5 MB, anything that does not decode as an image, and non-PNG formats (the provided tiles are PNG); convert to RGB
2. Hash + dedupe: SHA-256 of the file bytes. If hash + model_version already exists, return the stored result (idempotent retries).
3. Preprocess: the transform used at training time (resize, RGB order, normalization).
4. Classify: get probabilities for all 7 classes.
5. Confidence policy: marking auto_accepted or needs_review by threshold.
6. Store: write one row and save the raw tile to disk.
7. Query: analyst reads results through the API

Storage and querying:

I am using a SQLite table. It is a single file, needs zero setup, works offline and is plenty for this volume.

Column	Why needed
tile_hash + model_version (unique)	Dedupe key; lets a new model re-classify the same tile
Filename, tile_path	Trace back to the raw image
Predicted_label, confidence	Answer and how sure the model is
Probabilities (JSON, all 7)	Re-threshold later without re-running the model
status	Auto_accepted/needs_review
Created_at	Time based queries and monitoring
Latency_ms	Cheap health signal

Querying:

- Filter: GET /results?label=&status=&min_conf=&max_conf=
- Look up one tile: GET /results/{tile_hash}
- Aggregate: GET /stats (returns counts and average confidence per class, and counts by status: auto_accepted vs needs_review)

NOTE: not using spatial queries because tiles carry no geo-coordinates.

Key decisions and trade-offs:

1. Model choice: frozen pretrained CNN plus logistic regression

Options considered	pros	cons
Zero-shot CLIP	No training	Large model; weak on 64x64 satellite imagery
Frozen ResNet18 features+logistic regression	Trains in minutes on CPU, small, easy to explain and retrain	Features come from ImageNet not satellite data
Full fine tuning	Best accuracy	Slow on CPU; harder to reproduce

Chose the middle option. CPU speed, explainability and easy retraining matter more than squeezing out accuracy. The results backed this up: 93.3% on a validation split and 92.9% on the held-out eval set. Fine-tuning would

likely add a few points (mostly on River/Highway), but I would only do it if the cost of errors justified the extra CPU time and complexity.

2. Low confidence predictions: storing and flagging
 - Reject (not storing): loses data and analyst can't see what was attempted.
 - Store as "unknown": hides model's best guess
 - Store with a needs_review flag: this is what I am doing. Keeps everything and analyst decides how to use.

Threshold is not a guess. I measured coverage against accuracy on the eval set (210 tiles, 15 errors in total):

Threshold	Auto-accepted	Accuracy on accepted	Sent to review	Errors caught (of 15)
none	100%	92.9%	0	0
0.70	94.3%	95.5%	12	6
0.90	87.6%	97.3%	26	10
0.95	83.8%	98.9%	34	13

I chose 0.90 as the default: 87.6% of tiles are auto-accepted at 97.3% accuracy, reviewers check about 1 in 8 tiles, and two-thirds of the mistakes are caught. If mistakes turn out to be expensive, 0.95 lets only 2 errors through at the cost of 16% review. The threshold is an environment variable, and because I store all probabilities it can change later without reprocessing.

This works because wrong predictions are less confident on average (0.79 vs 0.97 for correct ones). Limit: some errors are confidently wrong; per-class thresholds, a top-2 margin rule or calibration are next options.

3. Store: beyond the label, I store every probability, the model version, and the raw tile. The raw tile costs some disk but makes debugging and relabeling possible. If disk becomes a constraint, I would keep only the hash and path.
4. Sync vs async: the endpoint is synchronous: a 64x64 tile classifies in milliseconds on CPU. A queue with workers would only be worth it for larger batches or bigger models.
5. Offline: model weights are bundled with the code. The service never downloads anything at runtime and.

Evaluation results: I trained on candidate_tiles/ (80/20 stratified split as a sanity check, then refit on all of it) and measured on the 210-tile eval set against eval_labels.csv; the eval set was never used for training. Accuracy is 93.3% on validation and 92.9% on the eval set, so the model generalizes rather than memorizes. The weakest classes are Highway and River (F1 0.87): most errors are River/Highway/AnnualCrop confusions, i.e. thin linear features crossing farmland. Caveat: 30 tiles per class means one tile moves a class score by about 3%, so per-class numbers and the threshold are provisional. The eval set stays frozen as a canary set for health checks.

Assumptions:

- Tiles are always RGB, 64x64, same format as provided in EuroSAT subset.
- Each tile has exactly one land-use label, from the 7 classes only.
- Volume is modest (thousands of tiles, not millions), so a single process with SQLite is enough
- One machine runs everything on the CPU with no GPU and no network.
- Wrong confident label is worse for the analyst than an honest "needs review".
- Analysts are trusted internal users, so authentication is out of scope for the slice.

Questions to ask:

1. Do tiles carry coordinates, capture time, or sensor ID? Then spatial and time queries are main features.
2. Will SAR or multispectral bands arrive later? This changes the model input.
3. Which mistakes hurt the most? Is missing industrial worse than a false forest? This sets thresholds per class.
4. What decisions are being made from these results? Would they review flagged tiles themselves?
5. How many tiles per day? What CPU and RAM does the offline box have?
6. How does a new model version reach an offline machine? Should old tiles be reclassified?
7. What happens to tiles like clouds or bare soil that fit none of the 7 classes?